

Chapter 4

Why do we need BaseEntity?

Without `BaseEntity`:

```
User
├── id
├── createdAt
├── updatedAt

Organization
├── id
├── createdAt
├── updatedAt

Role
├── id
├── createdAt
├── updatedAt
```

Lots of duplicated code.

With `BaseEntity`:

```
graph TD
    BaseEntity --> User
    BaseEntity --> Organization
    BaseEntity --> Role
```

Every entity automatically gets the common fields.

Fields in BaseEntity

Field	Purpose
id	Unique identifier (UUID)
createdAt	Record creation time
updatedAt	Last update time
createdBy	User who created the record
updatedBy	User who last updated it
version	Optimistic locking (prevents concurrent update conflicts)

Annotations Learned

Annotation	Purpose
@MappedSuperclass	Shares fields with child entities without creating a table
@Id	Marks the primary key
@GeneratedValue(strategy = GenerationType.UUID)	Automatically generates UUIDs
@Column	Maps Java fields to database columns
@Version	Enables optimistic locking
@Getter (Lombok)	Generates getter methods
@Setter (Lombok)	Generates setter methods

Lombok

Without Lombok:

```
Java
public String getName(){...}

public void setName(String name){...}
```

With Lombok:

```
Java
@Getter
@Setter
```

Lombok automatically generates getters and setters during compilation.

Best Practice

- ✓ Use @Getter and @Setter for JPA entities.
- ✗ Avoid @Data on JPA entities.

UUID

Instead of

1 , 2 , 3

We use

550e8400-e29b-41d4-a716-446655440000

Benefits

- More secure
- Hard to guess
- Better for distributed systems
- Good for APIs and future microservices

@MappedSuperclass

@MappedSuperclass

`BaseEntity` is not a database table.

It only provides fields to child entities.

Example:

`</> Java`

```
public class User extends BaseEntity {  
}
```

Hibernate treats `User` as if it contains:

id
created_at
updated_at
created_by
updated_by
version

Project Structure

```
common/  
└─ entity/  
    └─ BaseEntity.java
```

Quick Interview Notes

Question	Answer
Why use BaseEntity?	To centralize common fields (ID, audit fields, version) and avoid code duplication.
Why use @MappedSuperclass instead of @Entity?	Because BaseEntity is not a standalone table; it only shares its fields with child entities.
Why use UUID instead of Long IDs?	Better security, non-sequential IDs, and easier support for distributed systems and microservices.
Why use @Version?	To implement optimistic locking and prevent lost updates when multiple users modify the same record.
Why use Lombok?	To automatically generate boilerplate code such as getters and setters, making the code cleaner and easier to maintain.